DUNGEON RUN

Game Design Document (GDD)

Versión: 1.0 — Diseño definitivo inicial
Género: Arcade 2D, exploración, evasión y recolección
Perspectiva: Vista cenital (*top-down*)
Motor: Phaser
Lenguaje: JavaScript
Entorno: Node.js y Vite
Asistente de programación: OpenCode
Recursos visuales: Kenney Micro Roguelike

---

1. Concepto general

Dungeon Run es un juego arcade de exploración y evasión en 2D. El jugador recorre una mazmorra compuesta por habitaciones conectadas, recolecta reliquias y evita criaturas que lo persiguen.

Cada etapa presenta una distribución de habitaciones diferente. El jugador debe reunir seis reliquias para desbloquear la salida y avanzar a la siguiente etapa. La dificultad aumenta progresivamente.

La partida termina cuando el jugador pierde sus tres vidas. La puntuación obtenida se registra en una tabla local de mejores resultados, incentivando al jugador a volver a intentarlo y superar su récord.

El juego no incluye combate: los enemigos representan amenazas que deben esquivarse mediante el movimiento y la exploración.

2. Objetivo y ciclo de juego

El objetivo consiste en avanzar por la mayor cantidad de etapas posible y conseguir la puntuación más alta.

El ciclo principal es el siguiente:

1. Comenzar una partida con tres vidas.
2. Explorar las habitaciones y recolectar seis reliquias.
3. Evitar a los enemigos y utilizar el entorno para mantenerse a salvo.
4. Desbloquear la salida y avanzar a la siguiente etapa.
5. Repetir el proceso con una dificultad creciente.

6. Al perder las tres vidas, consultar la puntuación final y comenzar una nueva partida.

3. Mecánicas principales

3.1. Movimiento y colisiones

- Movimiento en cuatro direcciones mediante WASD o las flechas del teclado.
- Movimiento fluido y respuesta inmediata a los controles.
- Las paredes y los obstáculos sólidos impiden el paso.
- Las colisiones deben funcionar correctamente tanto para el jugador como para los enemigos.

3.2. Recolección de reliquias

- Cada etapa contiene seis reliquias.
- Al recoger una reliquia, esta desaparece del escenario.
- El contador de reliquias y la puntuación se actualizan automáticamente.
- La salida permanece bloqueada hasta que se recolecten las seis reliquias.
- Las reliquias deben ubicarse en posiciones accesibles.

3.3. Enemigos

- Los enemigos permanecen en una zona o patrullan dentro de su habitación.
- Cuando el jugador entra en su radio de detección, comienzan a perseguirlo.
- Si el jugador se aleja lo suficiente, el enemigo deja de perseguirlo y regresa a su comportamiento inicial.
- Los enemigos no pueden atravesar paredes ni obstáculos.
- La persecución se limita inicialmente a la habitación en la que se encuentra el enemigo, evitando implementar navegación compleja entre habitaciones.
- El jugador no puede atacar ni eliminar enemigos.

3.4. Vidas, daño e invulnerabilidad

- El jugador dispone de tres vidas para toda la partida.
- El contacto con un enemigo resta una vida.
- Tras recibir daño, el jugador obtiene un breve período de invulnerabilidad.
- Un efecto visual sencillo indica cuándo el jugador está protegido.
- Si todavía quedan vidas, el jugador reaparece en una posición segura de la habitación inicial de la etapa actual.
- Al perder las tres vidas, la partida termina y se muestra la pantalla de resultados.

Las vidas no se recuperan al avanzar de etapa.

3.5. Salida y progresión

- Cada etapa contiene una salida.
- La salida se desbloquea al reunir las seis reliquias.

- Al alcanzar la salida desbloqueada, el jugador avanza a una nueva etapa.
- Las etapas posteriores presentan desafíos progresivamente mayores.
- La progresión continúa hasta que el jugador pierde todas sus vidas.

4. Puntuación y tabla de récords

La puntuación recompensa la exploración y la supervivencia.

- Cada reliquia recolectada otorga puntos.
- Completar una etapa otorga una bonificación.
- Al finalizar la partida se muestra la puntuación total.
- El juego incluye una tabla local con las mejores puntuaciones.

Los récords se almacenan en el navegador mediante almacenamiento local. No se requiere crear cuentas ni utilizar servidores externos.

La tabla es local al dispositivo y navegador: no sincroniza resultados entre diferentes jugadores o equipos.

Los valores exactos de puntuación se determinarán durante las pruebas de balance.

5. Diseño y generación de niveles

5.1. Habitaciones modulares

La mazmorra se construye combinando habitaciones diseñadas previamente.

Cada habitación cuenta con una estructura definida, obstáculos y puntos de conexión compatibles con otras habitaciones. El sistema combina estas plantillas para generar distribuciones diferentes.

La generación será modular y controlada, no una creación completamente aleatoria de paredes y pasillos.

5.2. Reglas de generación

El sistema deberá garantizar que:

- Las habitaciones relevantes sean alcanzables desde la habitación inicial.
- Las conexiones entre habitaciones sean válidas.
- Las seis reliquias de cada etapa sean accesibles.
- La salida no quede bloqueada ni aislada.
- El jugador y los enemigos no aparezcan dentro de paredes u obstáculos.
- La generación no coloque elementos esenciales en posiciones inaccesibles.

La implementación comenzará con un conjunto reducido de habitaciones comprobadas. La variedad se ampliará después de validar el funcionamiento básico.

5.3. Dificultad progresiva

La dificultad podrá aumentar mediante:

- Una mayor cantidad de enemigos.
- Una distribución más exigente de las reliquias.
- Habitaciones con obstáculos que requieran mayor atención al desplazarse.

Los desafíos deberán ser compatibles con las mecánicas existentes y no depender de combate ni de inteligencia artificial avanzada.

6. Interfaz y controles

6.1. HUD

La interfaz mostrará:

- Vidas restantes.
- Reliquias recolectadas de un total de seis.
- Puntuación actual.
- Estado de la salida, cuando corresponda.

6.2. Pantallas

El juego contará con:

- **Menú inicial:** permite comenzar una partida.
- **Pantalla de juego:** muestra la mazmorra y el HUD.
- **Pantalla de fin de partida:** presenta la puntuación final, los mejores resultados y la opción de volver a jugar.

6.3. Controles

- WASD o flechas: movimiento.
- Interacción con la salida: avanzar cuando esté desbloqueada.
- Botones de interfaz: iniciar una partida o comenzar otra tras la derrota.

7. Dirección artística y audio

Se utilizará el paquete Kenney Micro Roguelike como base visual para el escenario, los personajes y los objetos.

Se priorizarán los recursos existentes y las animaciones incluidas en el paquete, evitando depender de ilustraciones o animaciones personalizadas complejas.

Se podrán incorporar efectos visuales sencillos para recoger reliquias, recibir daño y completar etapas.

Los efectos de sonido libres son deseables, pero tendrán menor prioridad que la funcionalidad principal.

8. Tecnologías y arquitectura

El proyecto utilizará:

- **JavaScript:** lógica del juego.
- **Phaser:** motor 2D, escenas, entrada, renderizado y colisiones.
- **Node.js y npm:** entorno de ejecución y administración de dependencias.
- **Vite:** servidor de desarrollo y compilación.
- **OpenCode:** asistencia para planificar, implementar y probar cambios.
- **Visual Studio Code:** editor principal.

La estructura prevista será:

- `src/scenes/`: escenas de juego, menú y pantalla de resultados.
- `src/entities/`: jugador y enemigos.
- `src/systems/`: vidas, puntuación, reliquias, progresión y generación.
- `src/patterns/`: plantillas de habitaciones.
- `src/assets/`: recursos gráficos y sonoros.
- `src/main.js`: punto de entrada del juego.

La estructura podrá adaptarse si existe una razón técnica clara, manteniendo el código organizado y comprensible.

9. Alcance de la primera versión

Incluido

- Movimiento del jugador y colisiones.
- Habitaciones conectadas.
- Generación modular mediante plantillas.
- Seis reliquias por etapa.
- Enemigos con detección y persecución limitada.
- Tres vidas por partida.
- Daño e invulnerabilidad temporal.
- Salida desbloqueable y avance entre etapas.
- Dificultad creciente.
- Puntuación, derrota y reinicio.
- Tabla local de mejores puntuaciones.
- HUD y pantallas básicas.

Fuera del alcance inicial

- Combate, armas e inventarios complejos.

- Jefes y comportamientos avanzados de enemigos.
- Generación completamente aleatoria de geometría.
- Navegación de enemigos por toda la mazmorra.
- Multijugador y tablas de clasificación online.
- Cuentas de usuario.
- Guardado permanente del progreso de una partida.
- Historia extensa, diálogos y animaciones originales complejas.

10. Plan de desarrollo con OpenCode

El proyecto se desarrollará mediante pasos pequeños y verificables.

1. Preparar el entorno y crear el proyecto con Phaser y Vite.
2. Comprobar que el proyecto se ejecute correctamente.
3. Implementar una habitación de prueba, movimiento y colisiones.
4. Añadir reliquias, contador, puntuación y desbloqueo de salida.
5. Implementar enemigos, detección, persecución y daño.
6. Completar el sistema de vidas, invulnerabilidad, derrota y reinicio.
7. Conectar habitaciones e implementar la generación modular.
8. Añadir progresión entre etapas y dificultad creciente.
9. Implementar y probar la tabla local de récords.
10. Corregir errores, pulir la interfaz y verificar los criterios de finalización.
11. Preparar el repositorio, la documentación y las evidencias de uso de OpenCode.

Antes de implementar cada cambio, OpenCode deberá proponer un plan concreto para revisión. Cada paso se probará antes de avanzar al siguiente. Los cambios importantes quedarán registrados mediante commits.

11. Criterios de finalización

La primera versión se considerará funcional cuando:

- El proyecto se instale y ejecute siguiendo las instrucciones.
- El jugador pueda desplazarse y las colisiones funcionen.
- Las habitaciones conectadas sean transitables.
- Se puedan recolectar seis reliquias y desbloquear la salida.
- Los enemigos detecten y persigan al jugador según las reglas definidas.
- El daño reste vidas y la invulnerabilidad evite daños repetidos inmediatos.
- Al perder las tres vidas, termine la partida y se muestre la puntuación.
- El jugador pueda comenzar una nueva partida.
- Se pueda avanzar entre etapas con una dificultad creciente.
- La generación no produzca elementos esenciales inaccesibles.
- Los mejores resultados permanezcan guardados al cerrar y abrir el navegador.
- El repositorio público incluya el código organizado, el GDD en PDF, `AGENTS.md`, `README.md` y las evidencias solicitadas.

12. Prioridades y decisiones de alcance

La prioridad es entregar un juego pequeño, completo y comprobable.

Primero se implementará una partida funcional con una habitación de prueba. Después se añadirán las habitaciones conectadas y la generación modular. Finalmente se incorporarán la progresión, la dificultad creciente y la tabla local de récords.

La cantidad de habitaciones, los valores de puntuación, el radio de detección, la duración de la invulnerabilidad y los parámetros de dificultad se ajustarán durante las pruebas.

Si el tiempo disponible obliga a reducir el alcance, se priorizará una experiencia jugable completa por encima de sistemas sofisticados que no lleguen a funcionar de forma fiable.